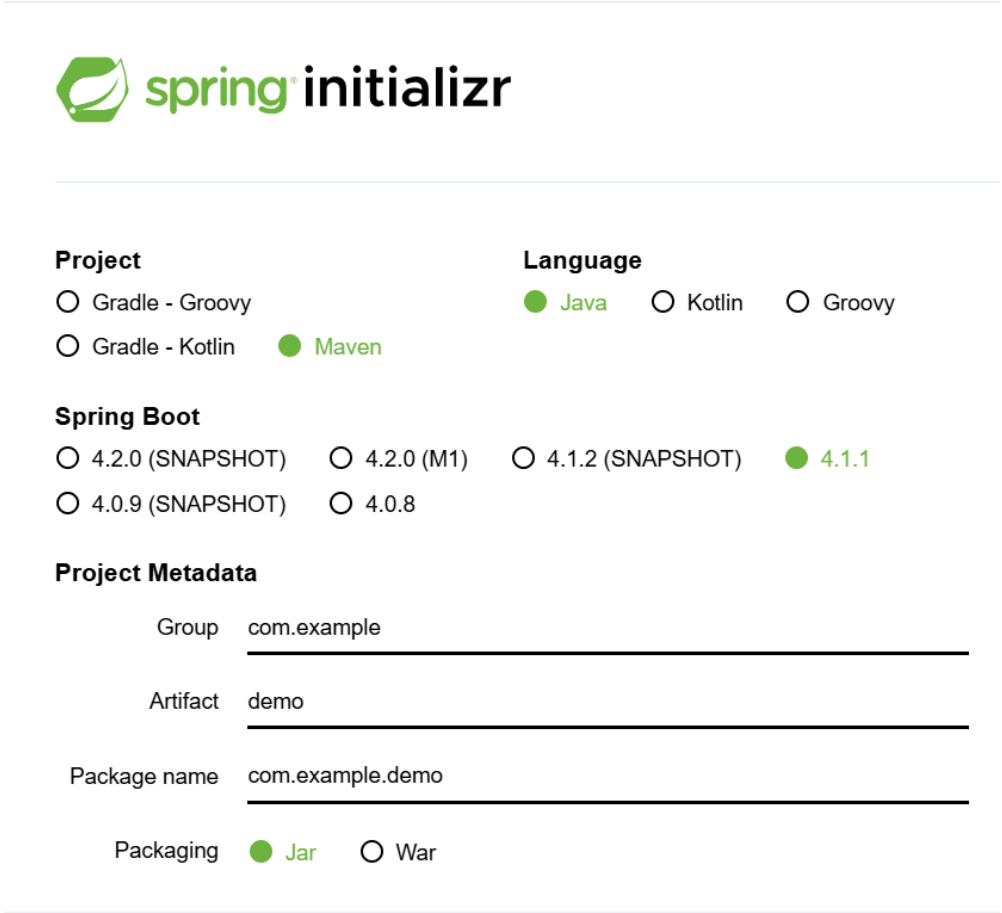
Building User Service

1. Create the project using Spring Initializr

Go to Spring Initializr - https://start.spring.io/

Setting	Selection
Project	Maven
Language	Java
Spring Boot	Latest non-SNAPSHOT version
Packaging	JAR
Java	Latest available version



2. Project Metadata

Group

```
com.ecommerce
```

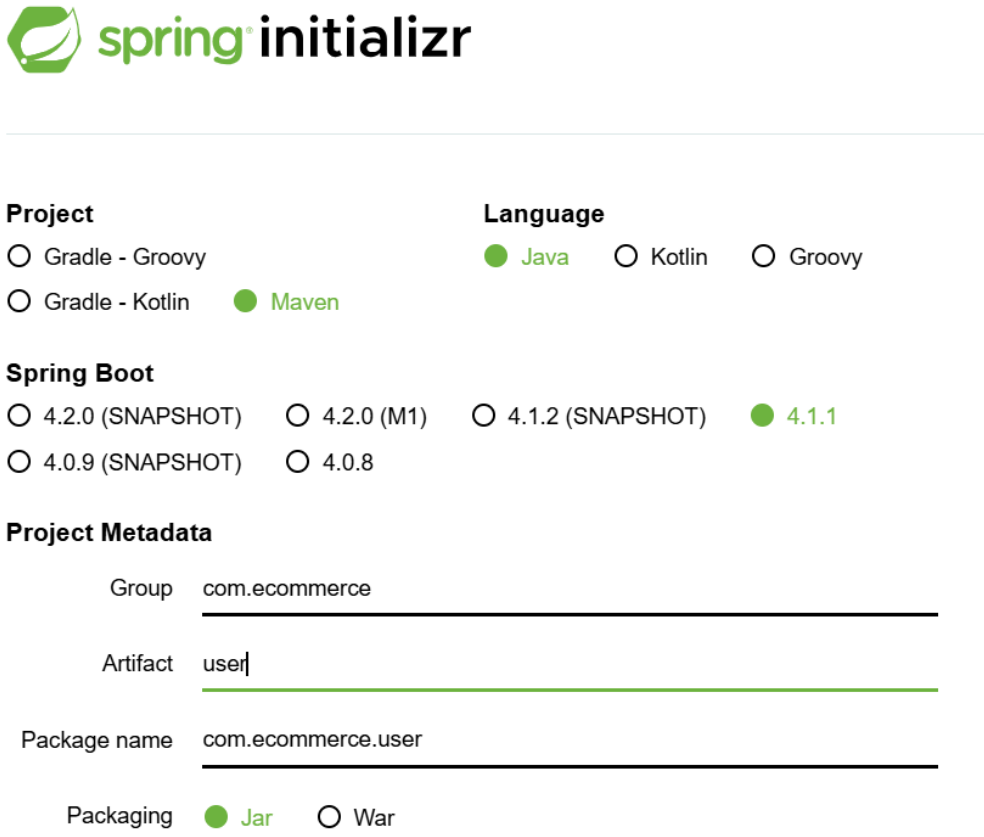
Artifact

```
user
```

This results in a project representing the **User Microservice**.

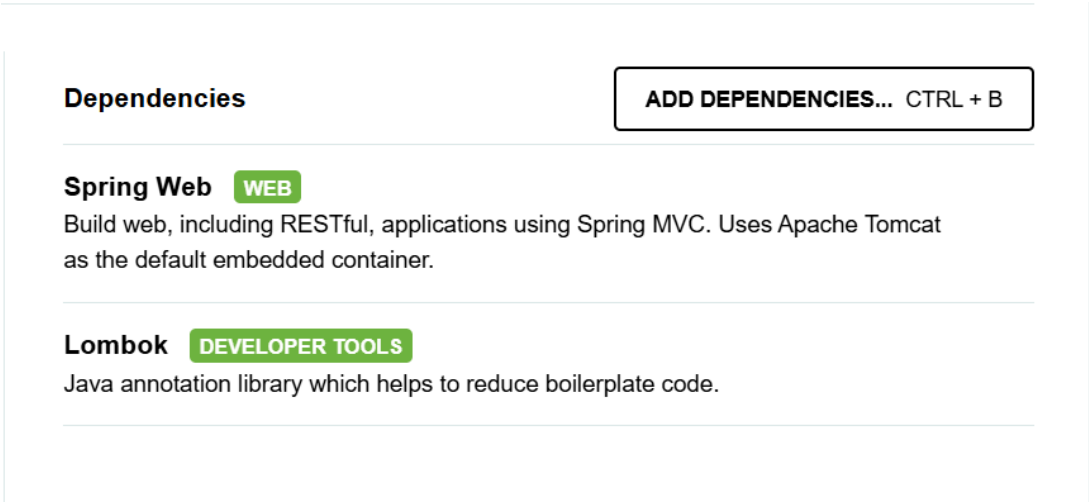
The description can be something like:

```
User service for e-commerce application
```



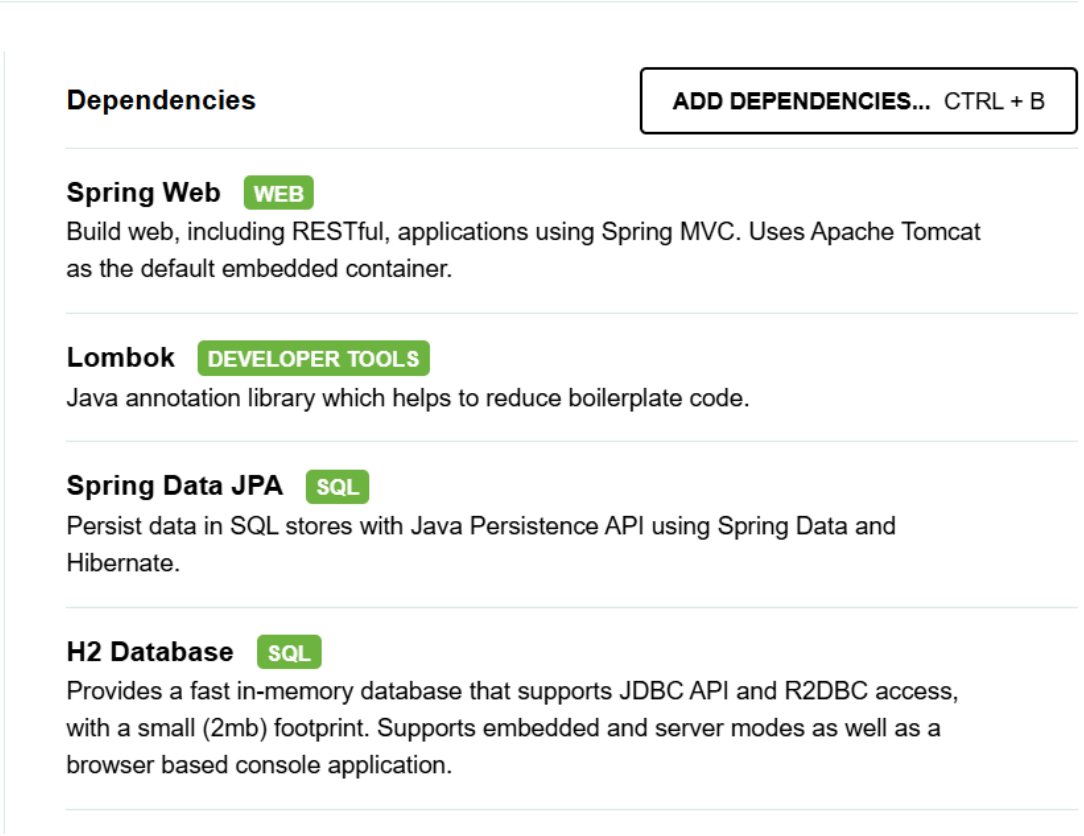
3. Dependencies- Web, Lombok

- The **User Service** needs several dependencies
- **Spring Web** – required because we will be building REST API's
- **Lombok** – helps reduce **boilerplate code**



4. Dependencies - Spring Data JPA

- JPA will be used for **database access and persistence**
- The course initially uses H2 database



5. Generate the Project

- Generate the project after selecting the required options
- Generate -> Download Zip
- Spring Initializr generates the complete Spring Boot project
- The Zip is then downloaded to the local machine.

These dependencies have specific purposes:

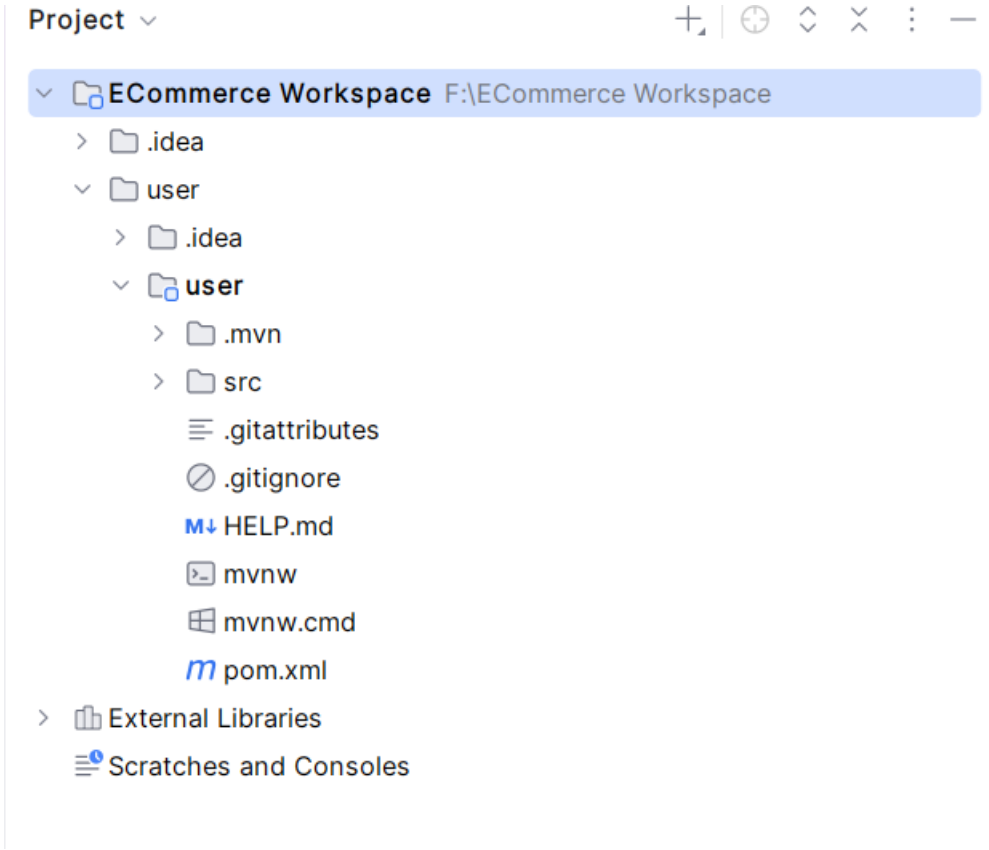
Dependency	Purpose
Spring Web	Build REST APIs
Lombok	Reduce boilerplate code
Spring Data JPA	Database persistence/access
H2	Initial database for the service

6. Extract the Project

- Extract the ZIP into the workspace
- The resulting structure

```
workspace/
│
├── user/
│   ├── src/
│   ├── pom.xml
│   └── ...
│
└── other projects...
```

The `user` directory represents our **User Microservice project**.



7. Open the User Service in IntelliJ

Open IntelliJ IDEA.

Choose:

```
Open
```

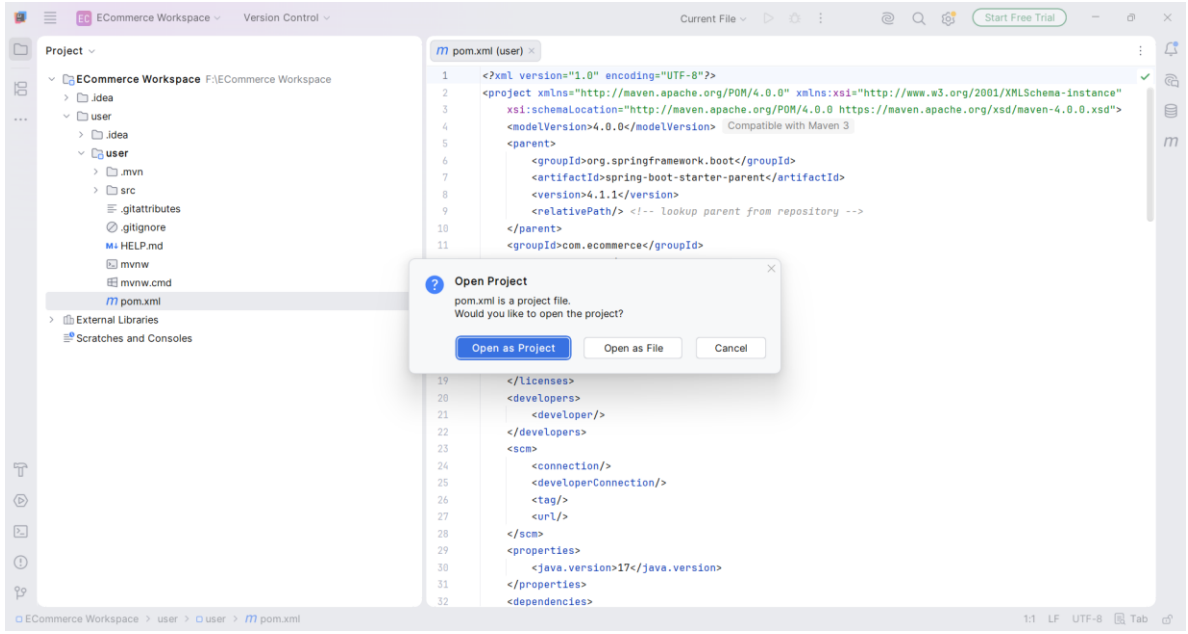
Navigate to the extracted `user` project.

Select:

```
pom.xml
```

Then choose:

> **Open as Project**



8. What Happens After Opening The Project

IntelliJ recognizes that this is a **Maven project** because of:

```
pom.xml
```

Maven reads the dependencies specified in the `pom.xml`.

For example:

```
pom.xml
   ↓
Read dependencies
   ↓
Maven downloads required libraries
   ↓
Project becomes ready
```

The IDE may show processing/downloading activity in the bottom-right corner.

This means Maven is resolving the project's dependencies.

9. Why is POM.XML Important ?

The `pom.xml` is the central configuration file for a Maven project.

It contains information such as:

- Project details
- Spring Boot configuration
- Dependencies
- Plugins
- Build configuration

For this project, the dependencies selected through Spring Initializr will appear in the `pom.xml`.